Interim Report 03

PROG 3300 – Integrated Project for Programming
IT Programming – Year 2

(SIRE)
Simulated Incident Response Environment
A Real-Time Training Platform for Emergency Scenarios

Prepared By:
~ *Nibble* ~
Tom Burchell
Leon Wasiliew
John Hay
Kael Payette

Project Sponsor:
Alfred Parks

Institution:
Nova Scotia Community College
IT Programming Program

Table of Contents

1.0 Introduction **4**
1.1 Purpose of the Document **4**
1.2 Project State **4**
1.3 Intended Audience **4**
2.0 System Implementation **5**
2.1 Coding Standards and Best Practices 5
2.2 Version Control and Collaboration 7
2.2.1 Tools Used and Rationale 7
2.2.2 Branching Strategy 7
2.2.3 Collaboration Workflow 9
2.3 Key Code Implementations 10
2.3.1 Authentication System 10
2.3.2 API Communication and Data Handling 10
2.3.3 Session Management System 12
2.3.4 Scenario Decision System 13
2.3.5 Reusable Components and Layout Architecture 13
2.3.6 Backend Logging and Security 14
2.3.7 Third-Party Libraries 14
2.4 Database Implementation 15
2.4.1 SQL Database for User Management 15
2.4.2 JSON for Scenario and Application Data 17
2.5 User Interface (UI) Design 19
2.5.1 Key UI Components 19
2.5.2 Reusable Components 19
2.5.3 User Interaction and Navigation 20
2.5.4 Design Approach 20
2.5.5 Visual Design 20

3.0 Testing **23**
3.1 Testing Types 23
3.1.1 Frontend 23
3.1.2 Backend 23
3.2 Test Cases and Results 23
3.2.1 Frontend 23
3.2.2 Backend 25
4.0 Summary **32**
5.0 References **33**

1.0 Introduction

1.1 Purpose of the Document

The purpose of Interim Report 03 is to present the current implementation progress of the Simulated Incident Response Environment (SIRE). This document outlines both non-technical and technical aspects of the system, including frontend and backend development. It provides an overview of the standards established, tools used, and tests conducted. It demonstrates the current state of the project before it moves into its final development/testing stage.

1.2 Project State

At this stage, Team Nibble has completed most of the implementation required for a static and near-functional prototype of the SIRE application. Core features, including user authentication, session management, and scenario interaction, have been developed and integrated across both ends.

The project is now approaching its final phase, with remaining work focused on refinement and testing in preparation for the Tech Showcase on March 26, 2026. The next steps planned include final UI improvements, bug fixing, and the introduction of automated testing through scripts or a Continuous Integration (CI) workflow.

NOTE: To support development, AI-assisted tools such as Microsoft Copilot and Google Gemini were used to generate and enhance visual assets, including the scenario images.

1.3 Intended Audience

- **Project Sponsor – Alfred Parks:** Reviews this report to assess project progress and confirm readiness for the final development stage.

2.0 System Implementation

2.1 Coding Standards and Best Practices

Category	Description	Example
Coding Standards	Rules that ensure code is written consistently across the entire project, including formatting, spacing, and syntax style.	**Frontend & Backend:** Consistent indentation formatting, and clear separation of JSX and logic in React components, as well as consistent function and object formatting in backend controllers.
Naming Conventions	Guidelines for naming variables, functions, components, and files in a clear and meaningful way.	**Frontend:** React components use PascalCase (e.g., HomeLayout), non-default functions and variables use camelCase (e.g., setScenarios), and CSS classes use kebab-case (e.g., home-container). **Backend:** Functions and variables follow camelCase naming (e.g., sessionController).
Project Structure	Organizing files into logical folders to separate concerns and improve maintainability.	**Frontend:** Organized into folders under src, including assets, components, context, hooks, layouts, navigation, screens, services, and utils. **Backend:** Organized into folders under src, including config, controllers, middleware, models, routes, scenarios, services, sockets, and utils.
Clean Code	Writing simple, readable code where each function or module has a single responsibility.	**Backend:** Validation logic is separated into utility functions instead of being embedded directly in controllers, keeping controller methods clean and focused.
WET Principle	Write Everything Twice (WET) Principle allows similar code to exist in multiple places when small differences make reuse impractical.	**Frontend:** Multiple layout or UI sections are written separately (e.g., CreateSession & JoinSession for sessions) to allow unique content and structure for each scenario.

DRY Principle	Don't Repeat Yourself (DRY) Principle avoids repetition by reusing logic or components across the application.	**Frontend:** Reusable components (e.g., buttons or layout elements) are used across multiple screens. **Backend:** Shared validation and normal normalization functions are reused across different controller methods.
Error Handling	Managing errors to prevent crashes and provide meaningful feedback to users and developers.	**Backend:** Controller methods use structured error handling (e.g., try-catch statements and conditional checks) and return appropriate HTTP status codes and messages when errors occur.
Frontend Best Practices	Recommended techniques for building maintainable and scalable React applications.	**Frontend:** Components are modular and separated from assets and styles, with modern external libraries used for UI functionality.
Backend Best Practices	Structuring backend logic in a modular and scalable way using the separation of concerns.	**Backend:** Controllers handle request/response logic, while services manage business logic, and utilities handle validation and helper functions.
API Design	Designing backend endpoints in a consistent and RESTful manner.	**Backend:** Endpoints follow REST principles, using routes for creating, retrieving, and deleting resources.
Comments & Documentation	Providing clear explanations for files, functions, and complex logic.	**Frontend & Backend:** Use of file-level, multi-line comments for description, along with function-level documentation and selective inline comments for complex logic.
Security & Validation	Ensuring inputs are validated and the application is protected from invalid or unsafe data.	**Frontend:** Form inputs include built-in HTML validation (e.g., required fields) to reduce invalid submissions. **Backend:** Request data is validated and normalized before processing, with invalid inputs rejected using appropriate responses.

2.2 Version Control and Collaboration

2.2.1 Tools Used and Rationale

Git & GitHub

GitHub served as our central platform for version control, documentation storage, and collaborative development. The repository "sire-simulator" was created at the start of the project, including a ".gitignore", a proper folder structure, and access permissions for all team members. This ensured consistent organization and prevented untracked or unnecessary files from entering the codebase.

The repository contains folders for backend, frontend, documentation, scenarios, and testing. This supports modular development and provides a clear separation of responsibilities.

Microsoft Teams

Teams was used for our day-to-day communication, quick updates, and coordination between in-person meetings. The Teams group chat "The Floor Horseman" has served as our default communication channel. It has supported rapid clarification of tasks, sharing of progress, and alignment across all aspects of the development work.

Jira Software

Jira was used for sprint planning, task tracking, and workflow visibility. A full Jira project was created with epics for Project Management, Documentation, Testing, Deployment, Frontend, Backend, and Scenarios. Each team member was assigned to relevant epics, and the Jira board was linked to the GitHub repository to enable automatic issue tracking and commit association.

2.2.2 Branching Strategy

The team adopted a structured branching template to maintain code quality and prevent conflicts. Both the "main" and "dev" branches require pull request (PR) reviews. No direct pushes are allowed:

- **main:** The stable branch containing finalized, working code or the final versions of documentation. Only merges that are approved by other teammates are allowed.
- **dev:** The integration branch where all feature branches are merged before being promoted to main. This branch contains active development work.
- **feature branches:** Created per task using the format:
    - feature/<developer>/<feature-name>

Examples include:

- feature/tom/socket-engine
- feature/leon/admin-dashboard
- feature/john/scenario-fire
- feature/kael/documentation

Commit standards were also defined early in the project. Good commits required clear, descriptive messages and followed this structure: **(feat/fix/etc/)** + **(name)** + **(description)**.

Types include:

Type	Meaning	When to Use	Example Commit Message
feat	New Feature	Adding a new feature	feat: Add login page
fix	Bug Fix	Fixing a bug	fix: Resolve crash issue
docs	Maintenance tasks	Updating docs or README	docs: Update user guide
chore	Code style	General cleanup or upkeep	chore: Reorganize project files
style	Code style changes	Formatting-only changes	style: fix indentation
refactor	Code refactoring	Improving code without changing behavior	refactor: optimize auth logic
test	Adding or updating	Writing or modifying tests	test: Add unit tests
perf	Performance improvements	Enhancing speed or efficiency	perf: Improve load time.
build	Build System	Modifying build tools or dependencies	build: update dependencies

2.2.3 Collaboration Workflow

The team followed a consistent workflow combining GitHub, Jira, and Teams:

- Tasks were created and assigned in Jira, grouped under epics such as Backend Architecture, Frontend UI, and Scenario Development.
- Developers created feature branches for their assigned tasks and pushed updates regularly.
- Pull requests were used to merge work into the dev branch, ensuring review and integration before reaching main.
- Teams was used to coordinate progress, clarify responsibilities, and resolve blockers quickly.
- In-person meetings (twice weekly) were used to review progress, plan next steps, and ensure alignment across all roles.

2.3 Key Code Implementations

2.3.1 Authentication System

The full-stack JavaScript application employs a client-server approach for authentication, where the frontend communicates with backend API endpoints for login and signup. On the frontend, this is handled through api.js, apiClient.js, and the Login and Signup screens.

When a user submits their credentials, the application sends a POST request to the backend. If the request is successful, an authentication token is stored in localStorage. This token is then automatically included in future requests through the Authorization header, allowing access to protected sources. The frontend also performs basic client-side validation, such as checking required fields, verifying email format, and confirming password matching. On the backend, authentication behavior is managed through configuration files like securityConfig.mjs, which define request headers, API key requirements, and validation rules.

2.3.2 API Communication and Data Handling

All API communication is handled through a centralized API client (apiClient.js). This approach avoids repeating request logic across components and ensures consistency.

The API client is responsible for:

- Attaching headers (including authentication tokens)
- Parsing responses safely
- Handling errors in a structured way

It supports multiple request types (GET, POST, PUT, DELETE) and ensures that both JSON and non-JSON responses are processed without breaking the application. When an error occurs, a structured error object is returned, making it easier to debug issues and provide clear feedback to the user.

React apiClient.js – Frontend Example

```
/**
 * Author: Leon Wasiliew
 * Last Update: 2026-03-21
 * Description: Handles HTTP requests, including base URL configuration,
authentication headers,
 * error handling and response parsing for authentication.
 */
```

```
// Base URL for API requests (configured via Vite environment variables)
const API_BASE = import.meta.env.VITE_API_BASE?.replace(/\/$/, "") ||
"http://localhost:8080/api";

/** Function that retrieves the stored authentication token from localStorage. */
function getAuthToken() {
    return localStorage.getItem("authToken");
}

/** Asynchronous function to make API requests. */
async function request(endpoint, options = {}) {
    const token = getAuthToken();

    // Merges default headers with any provided in options, and adds the
Authorization header
    const headers = {
        "Content-Type": "application/json",
        ...options.headers,
        ...(token ? { Authorization: `Bearer ${token}` } : {}),
    };

    // Makes the API request with credentials included for session management
    const response = await fetch(`${API_BASE}${endpoint}`, {
        credentials: "include",  // Includes cookies in requests for session
management
        ...options,
        headers,
    });

    const text = await response.text();  // Reads response as text to handle both
JSON and non-JSON responses

    // Parses response data safely
    let data = null;
    try {
        data = text ? JSON.parse(text) : null;
    }
    catch {
        data = text;  // Assigns fallback if response is not valid JSON
    }

    if (!response.ok) {
        const errorMessage = (typeof data === "object" && data?.message) ||
`${response.status} ${response.statusText}` || "Unknown API error";
```

```
        const error = new Error(errorMessage);  // Creates an Error object with the
message from the response
        error.status = response.status;         // Attaches the HTTP status code to
the error object
        error.data = data;                      // Attaches the response data to
the Error object for additional context
        throw error;                            // Throws the error to be caught by
the caller
    }

    return data;  // Returns the parsed response data to the caller
}

/** API client for making HTTP requests. */
export default {

    /** Method for making GET requests. */
    get(endpoint) {
        return request(endpoint, { method: "GET" });
    },
    /** Method for making POST requests. */
    post(endpoint, body) {
        return request(endpoint, { method: "POST", body: JSON.stringify(body) });
    },
    /** Method for making PUT requests. */
    put(endpoint, body) {
        return request(endpoint, { method: "PUT", body: JSON.stringify(body) });
    },
    /** Method for making DELETE requests. */
    delete(endpoint) {
        return request(endpoint, { method: "DELETE" });
    },
};
```

2.3.3 Session Management System

The application includes a session system that allows administrators to create and manage simulation sessions, while trainees can join and participate.

In the Admin Dashboard, the session lifecycle is controlled using React state, with three main stages: waiting, active, and ended. The interface updates dynamically based on the current state, displaying trainee participation, their stage, and final results.

The Create Session screen retrieves available scenarios from the backend and generates a session key. This key is used by trainees in the Join Session screen to access the session. While some elements currently use mock data, the overall structure is designed for full backend integration.

2.3.4 Scenario Decision System

The trainee interface is built around a decision-based system driven by structured scenario data. Each scenario consists of JSON nodes, and the application tracks the current node using state.

When a user selects an option, the application evaluates the outcome and either moves to another node or ends the scenario if a failure condition is reached. This data-driven approach separates logic from content, making the system easier to extend with new scenarios.

2.3.5 Reusable Components and Layout Architecture

The frontend is designed using reusable components and layout structures to maintain consistency across the application. For example, the Button component is used throughout the app and supports both navigation and custom actions.

Button – Component Example

```
/**
 * Author: Leon Wasiliew
 * Last Update: 2026-03-21
 * Description: Reusable button component displayed across the application.
 */

import { useNavigate } from "react-router-dom";
import "./Button.css";

/** Function that returns the Button component that can trigger navigation or
execute a custom action. */
export default function Button({ text, to, onClick, type = "button", ...rest }) {
    const navigate = useNavigate();

    /** Function that handles the button click event. */
    const handleClick = () => {
        if (to) navigate(to);
        if (onClick) onClick();
    };
```

```
    return (
        <button
        type={type}
        className="app-button"
        onClick={handleClick}
        {...rest}
        >
            {text}
        </button>
    );
}
```

Layout components such as FormLayout, HomeLayout, and AdminDashboardLayout provide consistent page structures while allowing individual screens to focus on their specific functionality. This separation improves readability, organization, and scalability.

2.3.6 Backend Logging and Security

The backend includes a structured logging and security setup. The audit logger (auditLogger.mjs) records important events while sanitizing input data to prevent unsafe values, such as prototype-related keys.

A centralized logger provides multiple log levels (info, warn, error, debug) with timestamps for better traceability. Configuration files like environmentConfig.mjs and securityConfig.mjs manage settings such as ports, headers, and API key requirements, making the system flexible across different environments.

2.3.7 Third-Party Libraries

The project uses several libraries to support core functionality:

- react-router-dom for client-side routing
- @mui/material for responsive layout design
- react-responsive-carousel for the homepage carousel

The backend relies mostly on custom-built modules but is structured in a way that allows additional libraries to be integrated if needed.

2.4 Database Implementation

This project utilizes a hybrid data storage approach by combining a relational SQL database with JSON (JavaScript Object Notation). This design was selected to balance the need for secure data management with the flexibility required for dynamic content.

The SQL database is used for managing user authentication and account data, while JSON is used to store scenario-based simulation data. This separation ensures that each type of data is handled using the most appropriate technology, improving both system efficiency and maintainability.

2.4.1 SQL Database for User Management

A relational SQL database is implemented to manage user accounts and authentication. This includes both Administrators and Trainees, as role selection occurs after login rather than being predefined within the database.

The database consists of a primary user table that stores the following attributes:

- Username
- Email address (unique)
- Password (hashed value)

This structure ensures that user data is stored securely and consistently. The use of a unique email constraint prevents duplicate accounts, while password hashing enhances security.

The SQL schema used for this implementation:

```
CREATE TABLE user (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL
);
```

This schema provides a simple but effective structure for user management. The id field is the primary key and uniquely identifies each user, while the email field enforces uniqueness to prevent duplicate registrations. The password_hash field ensures that plaintext passwords are never stored in the database.

Although simple, the design provides a foundation for future expansion.

Figure 1: SIRE Database Creation Screenshot

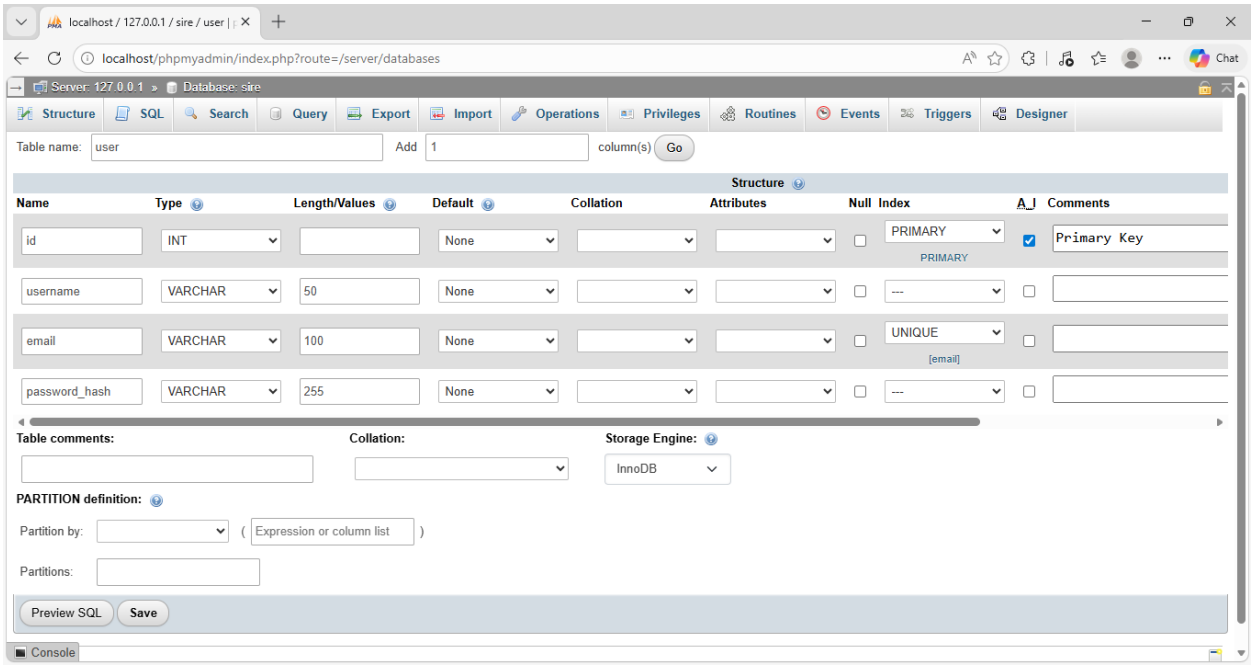
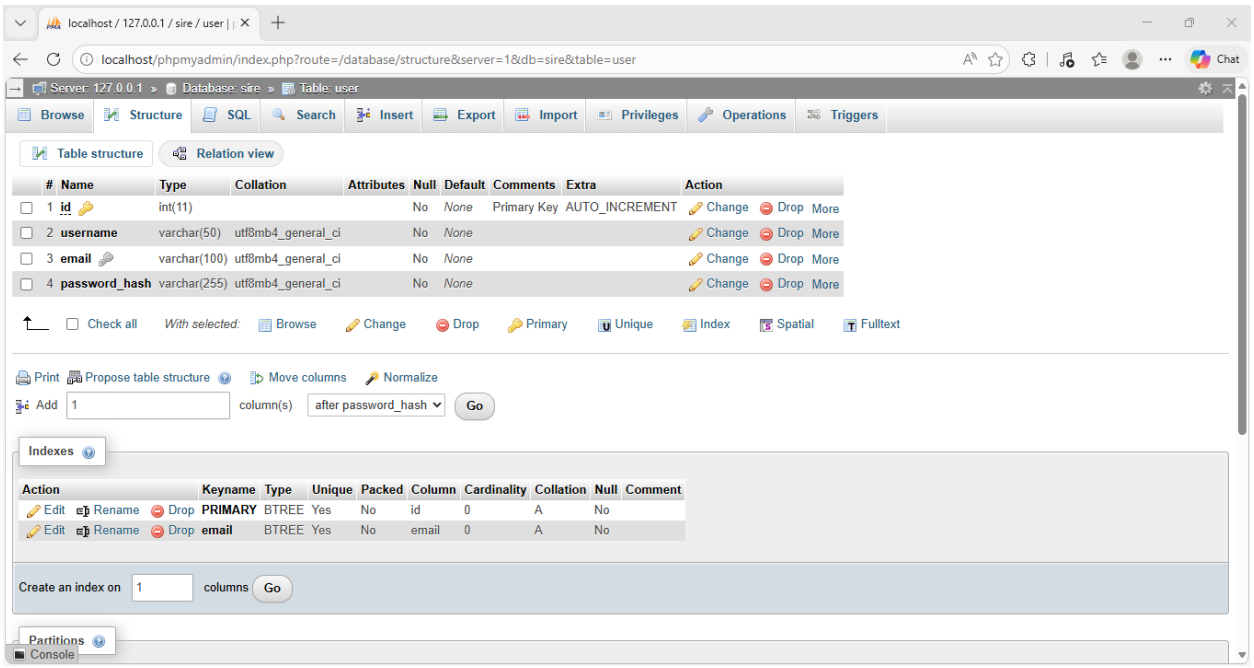


Figure 2: SIRE Database "user" Table Schema Screenshot



2.4.2 JSON for Scenario and Application Data

Scenario data and simulation logic are stored using JSON files. These files define the structure of each scenario, including decision nodes, possible actions, outcomes, and progression paths.

Class A Fire – JSON Example

```
{
  "schema": "1.0",
  "title": "CLASS A FIRE — FULL BRANCHING ADVENTURE TREE",
  "category": "Class A Fire",
  "root": "A1",
  "phases": [
    { "id": 1, "name": "Refined Core Tree" },
    { "id": 2, "name": "Deep Expansion" },
    { "id": 3, "name": "Human + Edge-Case Exhaustion" }
  ],
  "nodes": {
    "A1": {
      "phase": 1,
      "title": "NODE A1 — Initial Observation",
      "situation": "A faint smell of smoke is reported on the second floor. No
alarm is sounding. People nearby are unsure if it’s serious.",
      "question": "What do you do first?",
      "options": [
        {
          "label": "A",
          "text": "Ignore it until flames are visible",
          "outcome": { "type": "node", "target": "A1A", "note": "Delay" }
        },
        {
          "label": "B",
          "text": "Investigate the source of the smoke",
          "outcome": { "type": "node", "target": "A2", "note": "Correct
investigation path" }
        },
        {
          "label": "C",
          "text": "Pull the fire alarm immediately",
          "outcome": { "type": "node", "target": "A1C", "note": "Early alarm
path" }
        },
        {
          "label": "D",
```

```
          "text": "Call emergency services without checking",
          "outcome": { "type": "node", "target": "A1D", "note": "Premature
emergency call path" }
        }
      ]
    },
    "A1A": {
      "phase": 1,
      "title": "NODE A1A — Delay / Inaction",
      "situation": "Two minutes pass. The smell of smoke is stronger. Someone
nearby jokes that it’s “probably just someone burning popcorn.”",
      "question": "What do you do now?",
      "options": [
        { "label": "A", "text": "Keep waiting to avoid embarrassment",
"outcome": { "type": "node", "target": "A1A1", "note": "Critical delay" } },
        { "label": "B", "text": "Investigate the source immediately",
"outcome": { "type": "node", "target": "A2", "note": "Late recovery" } },
        { "label": "C", "text": "Pull the fire alarm", "outcome": { "type":
"node", "target": "A1C", "note": "Alarm, but late" } },
        { "label": "D", "text": "Ask someone else to check while you stay put",
"outcome": { "type": "node", "target": "A1A2", "note": "Delegation confusion" }
        }
      ]
    }, // ...
  }
}
```

2.5 User Interface (UI) Design

The user interface (UI) for this application is designed using a component-based architecture in React. The design follows a structured layout approach, ensuring that users can easily navigate between different parts of the system.

The interface supports two primary user roles (Administrators and Trainees) and provides tailored workflows for each, including session creation, session participation, and scenario interaction.

2.5.1 Key UI Components

Layout Components

The application uses multiple reusable layout components to maintain a consistent structure across different screens. These include:

- **HomeLayout:** A split-screen layout that introduces the application, provides navigation options, and displays a scenario carousel.
- **FormLayout:** Used for authentication screens such as login and signup, with a clean input-focused design.
- **CreateSessionLayout and JoinSessionLayout:** Provide structured interfaces for session management tasks.
- **AdminDashboardLayout:** Supports monitoring and controlling session flow for administrators.
- **TraineeInterfaceLayout:** Designed for real-time interaction with scenario content.

2.5.2 Reusable Components

Several reusable components are implemented to improve modularity and maintainability:

- **Button Component:** Supports both navigation and custom event handling.
- **Footer Component:** Provides consistent branding and layout closure across all screens.
- **Carousel Component:** Displays rotating scenario previews on the home screen.

This modular approach allows components to be reused across multiple screens, improving code modularity and scalability.

2.5.3 User Interaction and Navigation

Navigation within the application is handled using client-side routing. Each major screen is mapped to a route, allowing users to move between pages without full page reloads.

User interaction is designed to be straightforward:

- **Authentication Flow:** Users can sign up and log in using form-based inputs with validation and error feedback.
- **Role Selection:** Users choose between Administrator and Trainee roles, directing them to different workflows.
- **Session Management:** Administrators can create sessions and monitor trainee progress, while trainees can join sessions using a session key.
- **Scenario Interaction:** Trainees interact with scenario content by selecting options presented as clickable cards, which dynamically update the simulation state.

State management is handled using React hooks, allowing the interface to update dynamically in response to user actions.

2.5.4 Design Approach

The interface is designed with usability as a primary focus. Design decisions include:

- **Consistent Layouts:** All screens follow a similar split-screen or panel-based structure.
- **Clear Visual Hierarchy:** Important information, such as scenario content, is emphasized using spacing and card-based layouts.
- **Immediate Feedback:** Users receive feedback through loading states, error messages, success messages, and updated content when actions are performed.
- **Simple Navigation:** Logical workflows guide users through tasks without confusion.

2.5.5 Visual Design

The application uses a dark theme with white and red accent colors to highlight borders and important elements. Card-based layouts are used to group related information, particularly in the admin dashboard and the trainee interface.

Figure 3: Home Screen Screenshot

Figure 4: Create a Session Screen (With Validation) Screenshot

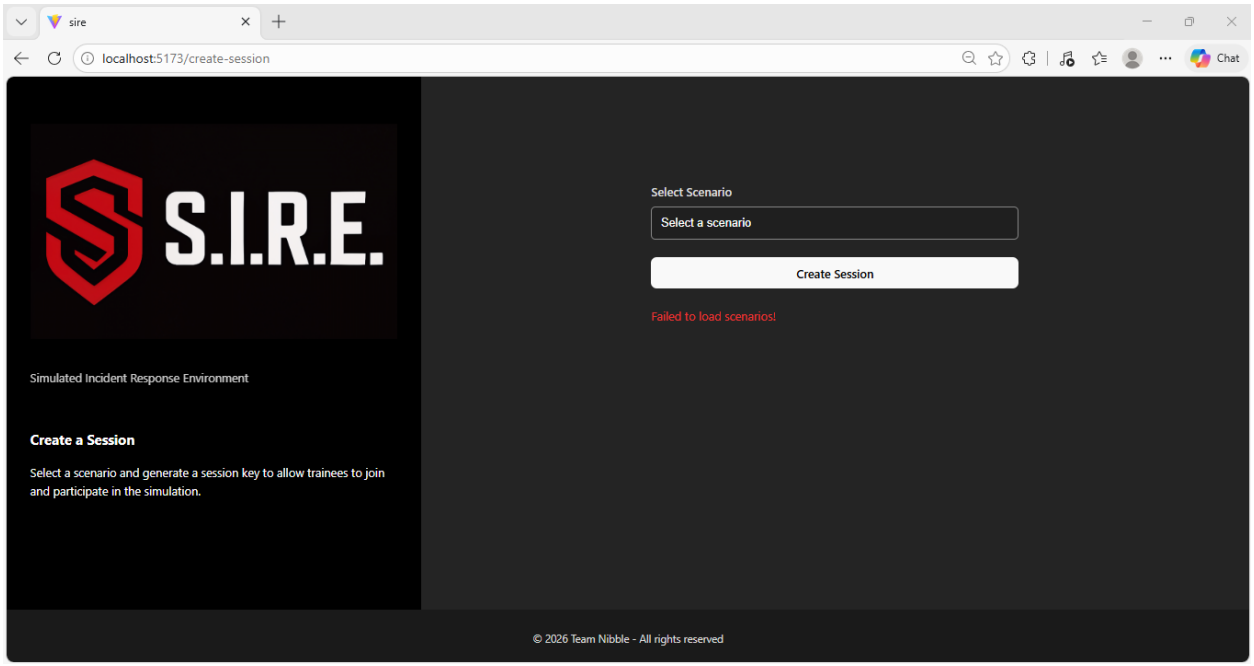


Figure 5: Administrator Dashboard (Before Session Start) Screenshot

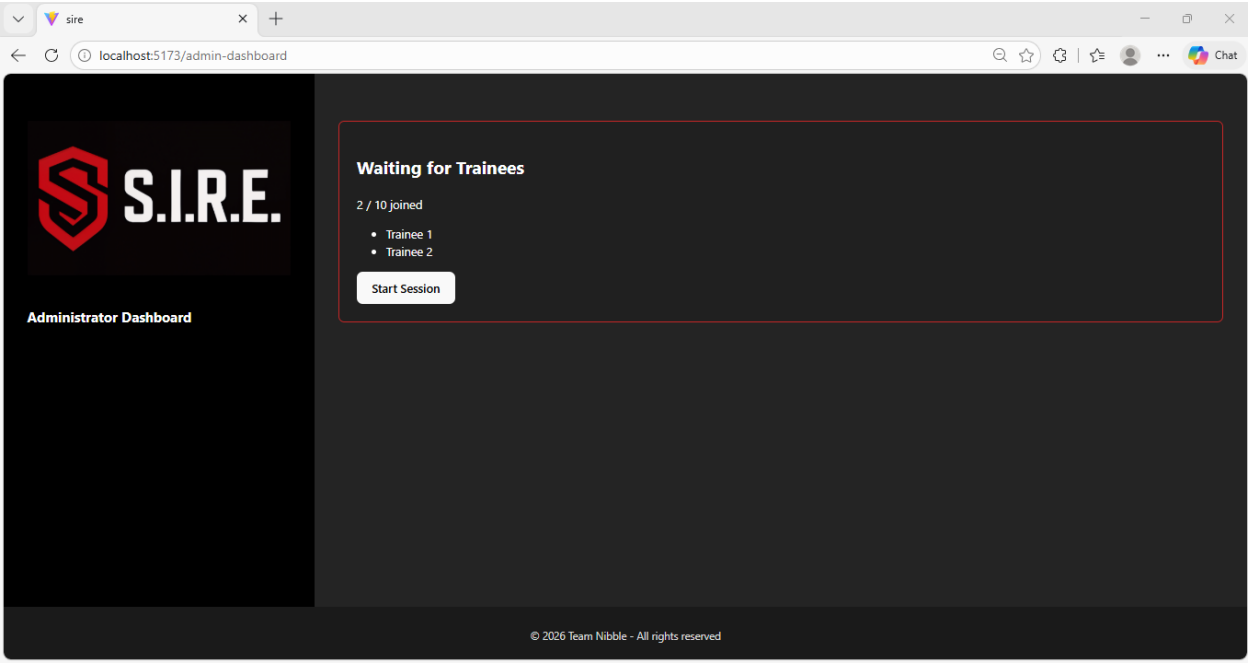
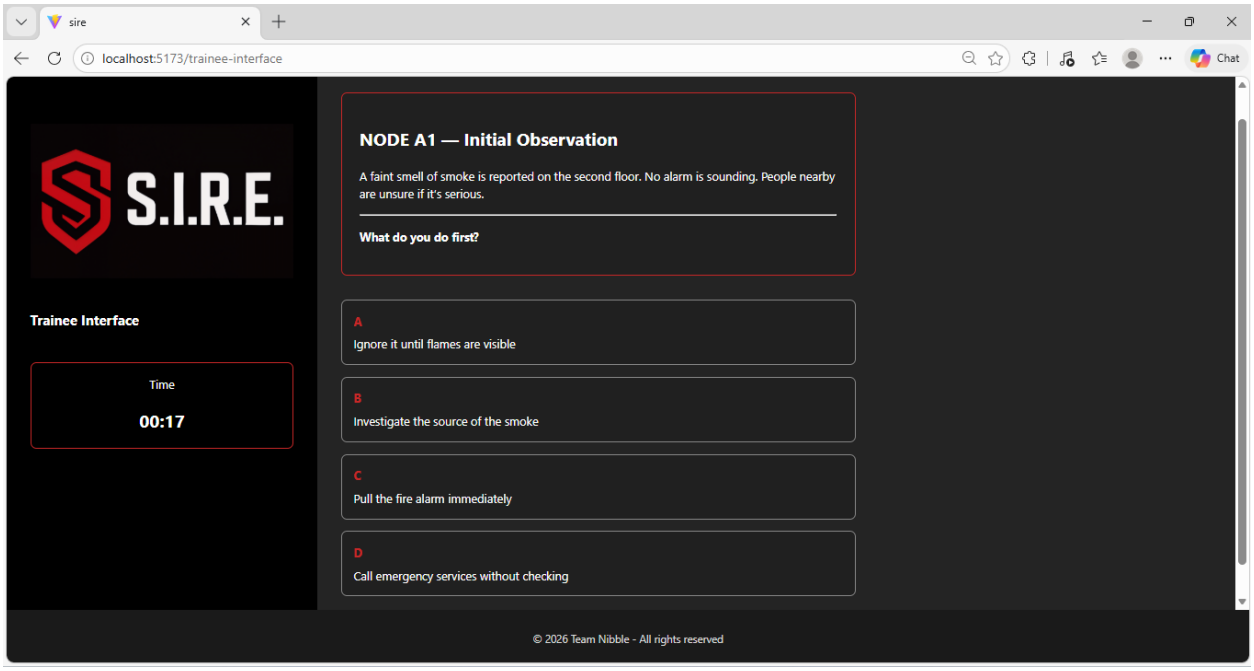


Figure 6: Trainee Interface (With Scenario) Screenshot



3.0 Testing

3.1 Testing Types

3.1.1 Frontend

The frontend was tested using a combination of manual and white-box testing techniques. Manual testing was used to verify UI layout responsiveness and navigation. White-box testing was applied during development to validate internal component logic, including form validation processes, control flow conditions, and state management within React components.

Automated testing has not yet been fully implemented, as integration between the frontend and backend is still in progress. Once the application is fully connected, automated testing tools such as Playwright may be introduced for end-to-end testing.

3.1.2 Backend

The backend was tested using a combination of integration testing and black-box testing techniques. Integration testing was used to verify that the individual server components, including REST API routes, authentication middleware, Socket.IO real-time communication, and session management, work correctly together as a complete system. Black-box testing was applied by sending HTTP requests and Socket.IO events to the running server and validating the responses, without relying on knowledge of the internal implementation.

Automated tests are written using Node.js's built-in assert module, with the socket.io-client library used to simulate real-time client connections. Tests cover two areas: server integration tests (verifying REST API and Socket.IO behaviour end-to-end) and CORS configuration tests (validating that cross-origin request handling is correctly enforced for both allowed and disallowed origins).

3.2 Test Cases and Results

3.2.1 Frontend

Resizing Test Cases

- Tested application across different screen sizes
- Adjusted height and width values to improve layout consistency
- Identified the need for additional refinement using minimum and maximum

Result: Layout is functional but requires minor improvements for full responsiveness

Navigation Test Cases

- Temporary navigation buttons were added to the home screen for testing
- Verified that all links correctly redirect to their intended pages
- Checked for broken or missing routes

Result: All navigation is functioning correctly, and no broken links or routing issues were found.

Frontend White-Box Test Case #1 – Internal Form Validation Logic

Test Case ID	FE-WB-01
Test Type	White-Box Testing
Component	src/screen/Login/index.jsx src/screen/Signup/index.jsx
Description	Verify that the internal validation logic is correctly implemented and checks user input before allowing form submission.
Preconditions	Form component is rendered and ready for input. Backend is not connected to the frontend.
Expected Result	Validation function prevents submission when inputs are invalid and allows submission only when inputs meet required conditions.
Actual Result	Form blocks invalid input and displays errors, while valid input successfully passes validation.
Status	Pass

3.2.2 Backend

Backend Integration Test Cases

Test Case ID	BE-IT-01
Test Type	Integration Testing / Black-Box Testing
Component	src/routes/sessionRoute.mjs src/middleware/authMiddleware.mjs
Description	Verify that the REST API rejects requests without a valid API key and returns HTTP 401 Unauthorized.
Preconditions	Backend server is running. No API key is provided in the request headers.
Expected Result	Server returns HTTP 401 Unauthorized for requests without a valid x-api-key header.
Actual Result	Server correctly returned HTTP 401 for all requests missing or using an invalid API key.
Status	Pass

Test Case ID	BE-IT-02
Test Type	Integration Testing / Black-Box Testing
Component	src/routes/sessionRoute.mjs src/services/sessionService.mjs
Description	Verify that an authenticated request to GET /api/session returns a list of active sessions, including a previously created session.
Preconditions	Backend server is running. A session has been created programmatically before the request. Valid API key is provided.

Expected Result	Server returns HTTP 200 with a JSON array of sessions. The pre-created session is present in the list.
Actual Result	Server returned HTTP 200 with a session array containing the expected session entry.
Status	Pass

Test Case ID	BE-IT-03
Test Type	Integration Testing / Black-Box Testing
Component	src/routes/sessionRoute.mjs src/utils/validation.mjs
Description	Verify that passing an invalid query parameter (e.g. limit=not-a-number) to GET /api/session returns HTTP 400 Bad Request.
Preconditions	Backend server is running. Valid API key is provided. Query string contains an invalid limit value.
Expected Result	Server returns HTTP 400 Bad Request due to invalid query parameter.
Actual Result	Server correctly returned HTTP 400 when an invalid limit value was provided.
Status	Pass

Test Case ID	BE-IT-04
Test Type	Integration Testing / Black-Box Testing
Component	src/sockets/socketServer.mjs src/middleware/authMiddleware.mjs
Description	Verify that a Socket.IO client can authenticate using the legacy extraHeaders method

	(backward-compatible path) and successfully join a session.
Preconditions	Backend server is running with Socket.IO. A session exists. API key is passed in extraHeaders.
Expected Result	Client connects successfully and receives a session:joined event with the correct session code.
Actual Result	Client connected and received session:joined with the expected session code and timeline index of -1.
Status	Pass

Test Case ID	BE-IT-05
Test Type	Integration Testing / Black-Box Testing
Component	src/sockets/socketServer.mjs src/middleware/authMiddleware.mjs
Description	Verify that a Socket.IO client can authenticate using the auth payload method (browser-safe primary path) and successfully join a session.
Preconditions	Backend server is running with Socket.IO. A session exists. API key is passed in the auth object.
Expected Result	Client connects successfully and receives a session:joined event with the correct session code.
Actual Result	Client connected and received session:joined with the expected session code.
Status	Pass

Test Case ID	BE-IT-06
Test Type	Integration Testing / Black-Box Testing
Component	src/sockets/socketServer.mjs src/middleware/authMiddleware.mjs
Description	Verify that a Socket.IO connection attempt using a wrong API key is rejected with an UNAUTHORIZED error.
Preconditions	Backend server is running with Socket.IO. An incorrect API key is used in the auth payload.
Expected Result	Connection is refused and the client receives a connect_error event containing the message UNAUTHORIZED.
Actual Result	Connection was refused and the connect_error event contained the UNAUTHORIZED message as expected.
Status	Pass

Test Case ID	BE-IT-07
Test Type	Integration Testing / Black-Box Testing
Component	src/routes/sessionRoute.mjs src/middleware/authMiddleware.mjs
Description	Verify that DELETE /api/session/:code requires both an API key and a ticket ID header, and that omitting the ticket ID returns HTTP 400.
Preconditions	Backend server is running. A session exists. API key is provided but ticket ID header is omitted.
Expected Result	Server returns HTTP 400 when the ticket ID header is missing.

Actual Result	Server correctly returned HTTP 400 when the x-ticket-id header was absent.
Status	Pass

Test Case ID	BE-IT-08
Test Type	Integration Testing / Black-Box Testing
Component	src/routes/sessionRoute.mjs src/services/sessionService.mjs
Description	Verify that DELETE /api/session/:code with both a valid API key and ticket ID successfully deletes the session and removes it from subsequent session list responses.
Preconditions	Backend server is running. A session exists. Both API key and ticket ID headers are provided.
Expected Result	Server returns HTTP 200 and the session no longer appears in a subsequent GET /api/session response.
Actual Result	Server returned HTTP 200 and the session was absent from the subsequent session list.
Status	Pass

Test Case ID	BE-CT-01
Test Type	Integration Testing / Configuration Testing
Component	src/config/environmentConfig.mjs
Description	Verify that the environmentConfig module correctly parses the ALLOWED_ORIGINS environment variable and exposes it as an array.

Preconditions	ALLOWED_ORIGINS environment variable is set. The environmentConfig module is imported.
Expected Result	The allowedOrigins property is an array.
Actual Result	allowedOrigins was confirmed to be an array.
Status	Pass

Test Case ID	BE-CT-02
Test Type	Integration Testing / Configuration Testing
Component	src/server.mjs (CORS middleware configuration)
Description	Verify that the CORS middleware returns correct headers for a request from an allowed origin, including credentials support.
Preconditions	Express app is configured with a restricted list of allowed origins. Request is sent from an allowed origin.
Expected Result	Response includes Access-Control-Allow-Origin matching the request origin and Access-Control-Allow-Credentials set to true.
Actual Result	Response correctly included Access-Control-Allow-Origin set to the allowed origin and credentials header set to true.
Status	Pass

Test Case ID	BE-CT-03
Test Type	Integration Testing / Configuration Testing
Component	src/server.mjs (CORS middleware configuration)

Description	Verify that the CORS middleware does not return CORS headers for requests from disallowed origins.
Preconditions	Express app is configured with a restricted list of allowed origins. Request is sent from a disallowed origin.
Expected Result	Response does not include an Access-Control-Allow-Origin header.
Actual Result	Response contained no Access-Control-Allow-Origin header for the disallowed origin.
Status	Pass

Test Case ID	BE-CT-04
Test Type	Integration Testing / Configuration Testing
Component	src/server.mjs (CORS middleware configuration)
Description	Verify that wildcard CORS configuration allows requests from any origin without setting credentials headers.
Preconditions	Express app is configured with wildcard CORS (origin: "*"). Request is sent from any arbitrary origin.
Expected Result	Response includes Access-Control-Allow-Origin set to * and does not include Access-Control-Allow-Credentials.
Actual Result	Response correctly returned Access-Control-Allow-Origin: * without a credentials header.
Status	Pass

4.0 Summary

Interim Report 03 documents the near completion of the Implementation phase for the Simulated Incident Response Environment (SIRE) project. During this phase, the team focused on developing both the frontend and backend components of the application, as well as applying established coding standards and best practices. These practices ensure that the application remains consistent, maintainable, and scalable as development continues.

Additionally, testing efforts were conducted across both the frontend and backend to validate functionality, security, and integration. Frontend testing covered UI responsiveness and navigation, while backend testing employed automated integration tests to verify REST API authentication, session management, real-time Socket.IO communication, and CORS configuration. All backend test cases passed, confirming the server handles both valid and invalid inputs correctly and that security controls such as API key enforcement and ticket ID validation behave as intended.

5.0 References

1. GeeksforGeeks. (2018, July 12). *White box Testing Software Engineering*. GeeksforGeeks. https://www.geeksforgeeks.org/software-testing/software-engineering-white-box-testing/
2. GeeksforGeeks. (2023, December 27). *React Tutorial*. GeeksforGeeks. https://www.geeksforgeeks.org/reactjs/react/
3. GeeksforGeeks. (2024, January 24). *Node.js Tutorial*. GeeksforGeeks. https://www.geeksforgeeks.org/node-js/nodejs/
4. GitHub Contributors. (n.d.). *Conventional Commits*. Conventional Commits. https://www.conventionalcommits.org/en/v1.0.0/
5. Google. (2024). *Chatbot AI - Powered by Multi-Model AI Technologies & AI Assistant*. Chatbotai.co. https://chatbotai.co/
6. Microsoft. (2026). *Software Development Life Cycle | Microsoft Power Automate*. Microsoft.com. https://www.microsoft.com/en-us/power-platform/topics/phases-of-the-software-development-lifecycle?msockid=1679ddde67bd62d92639cafb66d2631f
7. Microsoft Copilot. (2025). *Microsoft Copilot*. Microsoft Copilot; Microsoft. https://copilot.microsoft.com/
8. SimplyJavaScript. (2026). *JavaScript Coding Guidelines & Style Guide (2026 Best Practices)*. Simplyjavascript.com; SimplyJavaScript. https://simplyjavascript.com/guidelines.html
9. Superhuman. (2026). *Grammarly*. Grammarly.com; Grammarly. https://www.grammarly.com/?msockid=1679ddde67bd62d92639cafb66d2631f
10. W3Schools. (2019). *JavaScript Style Guide*. W3schools.com. https://www.w3schools.com/js/js_conventions.asp